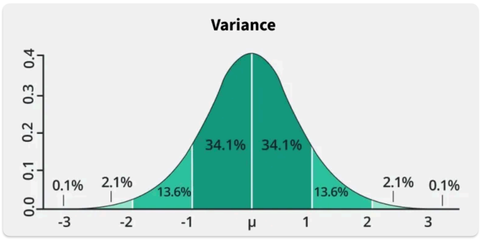
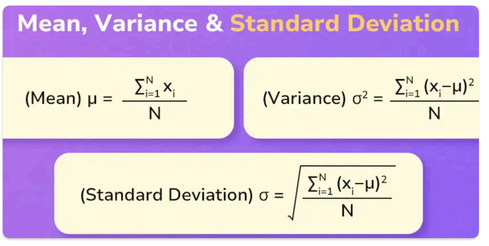
Maths for ML:

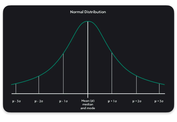
Mean(μ): Average of all values in a dataset
Median: Middle value in a dataset after the numbers are arranged in ascending or descending order.
Mode: Value that appears most frequently in a dataset
Range: The difference between the Max and Min value in a dataset
Variance(σ^2) : Variance is a number that tells us how spread out the values in a dataset are from the mean (average). It shows whether the numbers are closer to the average or far away from it.

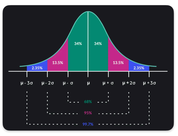
If the Variance is small, it means the most numbers are closer to the mean.
If the Variance is large, it means the numbers are spread out more widely.
A higher Variance indicates higher variability, meaning the data is spread, while a lower variance suggests the data points are closer to the mean.

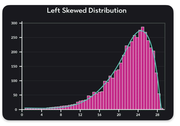
The following image depicts the variance in a Normal Distribution, illustrating how data points are spread around the mean(μ)

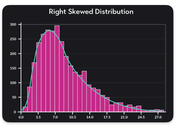


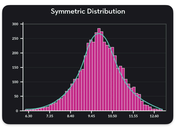




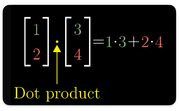


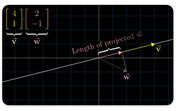


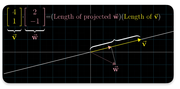


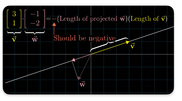


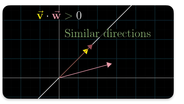
Vectors - Dot Product and Cosine Similarity

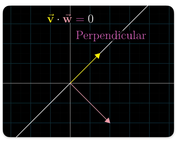


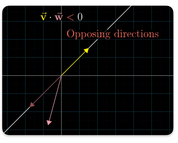




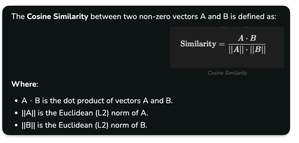


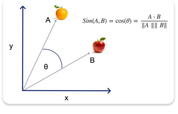












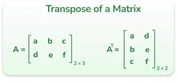
Matrix Multiplication













Gradient: The Gradient is fundamental concept in Calculus that extends the idea of a derivative to multiple dimensions. It plays an important role in Vector Calculus, optimization, Machine Learning, and Physics.
The Gradient of a function provides the direction of the steepest ascent, making it essential in areas such as Gradient Descent in Machine Learning and Optimization problems.

Mathematical Definition

Given a scalar function $f(x_1, x_2, \ldots, x_n)$ of multiple variables, the gradient is defined as a vector of its partial derivatives :

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right)$$

where each component $\frac{\partial f}{\partial x_i}$ represents the rate of change of f with respect to x_i.

Gradient in Two and Three Dimensions

For a function f(x,y), the gradient is:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

For a function f(x,y,z), the gradient is:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$$

Numerical Example

Consider the function:

$f(x, y) = x^2 + 3y^2$

Step 1: Compute the Gradient

The partial derivatives are:

$\frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 6y$

Thus, the gradient is:

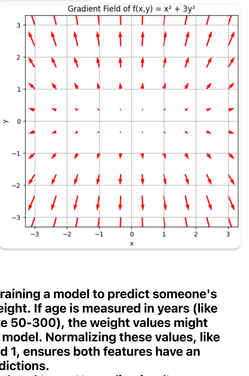
$\nabla f = (2x, 6y)$

Step 2: Evaluate at a Point

At (x,y)=(1,2):

$\nabla f(1, 2) = (2(1), 6(2)) = (2, 12)$

This means the function increases most rapidly in the direction ⟨2,12⟩.



Regularization: Imagine you're training a model to classify movie reviews as positive or negative. Without weight decay, the model might learn to strongly associate the word 'boring' with negative reviews, simply because it appeared in many negative reviews in the training data. With weight decay, the model is encouraged to consider other words as well, like 'uninspired' or 'predictable,' leading to a more balanced understanding of what makes a review negative.
In the context of large language models (LLMs), both dropout and weight decay serve as regularization techniques to prevent overfitting and improve model generalization.
Dropout: This technique randomly deactivates a fraction of neurons during training, which helps prevent the co-adaptation of neurons and promotes robustness. It introduces a form of ensemble learning, as the network trains on different subsets of neurons in each iteration.
Weight Decay: Also known as L2 regularization, this method penalizes large weights in the model by adding a term to the loss function proportional to the sum of the squared weights. It encourages a more balanced model by discouraging the model from assigning too much importance to any single input feature.
Both techniques are essential for optimizing LLMs, especially when dealing with the vast number of parameters in these models. The choice between dropout and weight decay often depends on the specific characteristics of the problem and the architecture of the neural network being used.

Normalization: Let's say you're training a model to predict someone's height based on their age and weight. If age is measured in years (like 0-100) and weight in pounds (like 50-300), the weight values might disproportionately influence the model. Normalizing these values, like scaling them to be between 0 and 1, ensures both features have an equal impact on the model's predictions.
Batch Normalization (Batch Norm) and Layer Normalization (Layer Norm) are techniques used to stabilize and accelerate the training of neural networks by normalizing activations. However, they differ in how and where they operate within the network.

Batch Normalization:
Batch normalization works on mini-batches of data and normalizes the activations across the batch dimension. For each feature in the mini-batch, it computes the mean and variance, then normalizes the activations using these statistics. This process reduces internal covariate shift, stabilizing training and enabling faster convergence.
Batch normalization is applied independently to each feature dimension (channel) and is particularly effective in feedforward and convolutional neural networks (CNNs) with large, consistent batch sizes. However, it depends on batch statistics, which can lead to performance issues during inference or with small batch sizes.

Layer Normalization:
Layer normalization, on the other hand, operates on individual samples and normalizes across the feature dimension for each layer. It computes the mean and variance for all features within a single sample, making it independent of batch size. This makes layer normalization more suitable for tasks like recurrent neural networks (RNNs) and transformer-based models, where batch sizes may vary or sequences have different lengths.
Layer normalization is robust to changes in input distribution and batch size, but it may converge slower compared to batch normalization in scenarios with large batch sizes.

Key Differences:
Scope of Normalization: Batch normalization normalizes across the batch dimension, while layer normalization normalizes across the feature dimension.
Dependency on Batch Size: Batch normalization requires sufficiently large batch sizes for stable training, whereas layer normalization is independent of batch size.
Use Cases: Batch normalization is commonly used in CNNs and feedforward networks, while layer normalization is preferred for RNNs, transformers, and scenarios with small or variable batch sizes.

Practical Implications:
The choice between batch and layer normalization depends on the architecture and dataset. Batch normalization excels in stabilizing training for large, consistent batch sizes, while layer normalization offers flexibility and stability for variable or small batch sizes.